Python for computational Problem Solving · Unit_02 · Unit 2: Collections & Functions

DRAFT — UE26CS151A · Unit_02 · Unit 2: Collections & Functions · Function Call Definition Variable arguments

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Function Call Definition Variable arguments
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Function Call Definition Variable arguments

1 Function Call Definition Variable arguments

Study focus: Function_Call_Definition_Variable_arguments. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000066 · EV0000067

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

I am Nita Anjaikan from the Department of Computer Science and Engineering.

Key details

- In this lecture, we will look at the concept of functions.
- We will look at how to define functions, what exactly functions are, and how we could call or use them.
- It may seem like a bunch of jargon to you, but let me put it in a simpler way.
- When you have a set of code that say would be used multiple times in your program, rather than copying or repeating those lines of code multiple times, I would rather club all those reusable lines together, put a name for it.
- Functions ideally would take an input, performs a specific set of operations, and computes and results gives you an output.
- We are actually familiar with this concept as we have seen in the first unit when we talked about your input and output functions.
- It takes an input, processes your input, does a couple of steps, and then returns an output.
- There are different kinds of functions available in Python.
- You have built in an existing functions like the print and input function, and there are user defined functions which as a programmer we can create based on the needs of our application.
- What are the advantages of using functions?
- They are, like I've already said, it reduces the duplication of code by just call the name of the function.
- It supports modularity, which means that every feature of the application that you build could be a function.

- It improves the clarity of code because it's so neatly structured as you will see in our examples that you can easily debug the code.
- It promotes reuse, it enhances readability, and the debugging and maintenance also becomes easier because when that there is a problem with a specific feature, you can directly jump to the lines of code where the function is defined and debug the code appropriately.
- So in our programming term when we say function definition, it means that we're going to try and create the function.
- Followed by the name of the function.
- A function name is an identifier, which means it should start with alphabets, then followed by a number or letter in English, or a digit, followed by a pair of round parentheses and then a colon.
- It is to give an input to the function if required.
- We will look at all of that in the upcoming lectures.
- Once we have and the leader with a colon, you can then use an indented block called the suite, which can have any valid statement in Python.
- It could be one or more statements, or it could also be another function, as we will see.
- As you can see, there is define the name of a function followed by a pair of parentheses and the parameters are optional.
- Then you have an indented block which would be the suite of the program.
- We must note that a function must be defined before we use or call the function.

Example or application from the lecture

- So first let's explore at what a function really is.
- there would be a leader and a suite because we are going to have multiple lines of code that are going to be clubbed and given a single name.
- So let's just look at an example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000066 · EV0000067 · EV0000068 · +1 source units in pack.json

Core explanation · 05:00–09:59

I know this is a redundant example, but it's just to show you how exactly it works.

Key details

- Every leader must end with a semicolon.
- Command prompt and I'm going to execute functions.py, which is the file in which I have written the code.
- You can see there is no output because all I have done is create the function.
- I have not brought the function to life.
- Followed by what we would call a function call operator.
- Invoke or we would activate the function.
- So when a function is defined, as you saw that when I executed, I did not get any output because the header or the leader is processed.
- By using the function call.

- At that point of time the control is shifted to the entire function block.
- The function is processed.
- And there was no output.
- It goes on to the next set of statements.
- Before display, I'm going to say print before function call.
- And after display print after function call.

Example or application from the lecture

- So let's look at another example where I'm going to like we've shown in this particular PPT, I'm going to create the same display function and have multiple print statements to show you what exactly would happen if I have other statements and what exactly the name of the function actually means.
- So this example is just to prove that.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000070 · EV0000071 · EV0000072 · +1 source units in pack.json

Core explanation · 10:00–14:59

The first two lines are not executed in the beginning.

Key details

- It's always going to be before and in between the before and after function call.
- You can see it says it is a function display at a particular location.
- This is because it's stored in the global memory, right?
- Which means your function is actually.
- just another object which has a property to be reused.
- What does this statement mean?
- Since the function is an object, I can assign one function object to another.
- I'm going to write display1 and the function call parameter.
- Heading over to the interpreter.
- So when you look at print display and print display one, you can see that both of them are functions at the same location, which means the name of a function is a function object or a variable which is basically a whose value is a function and that can be activated using a pair of round parentheses.
- Does this delete my function altogether?
- So if I were to now just call display in the next line, okay I'm going to save it.
- Head over to the terminal and execute it, you can say I get an error saying the name display is not defined.
- That's because I have deleted the display function here.
- After this I'm going to write display1 with the function call.
- Will this work?
- It works as expected.
- I can call it using either one of them.
- Associated with the function.

- It doesn't delete the function itself, which means I have display one also pointing to the same function.
- I hope you have understood the concept.
- In the upcoming videos, we will look at what exactly happens when a function is called, how we could give input to the function.
- and process these values as well.

Example or application from the lecture

- Let's look at it.
- Infer from this particular example is that display and display one are just names for that function object in the memory.
- So let's look at the next set of information in the upcoming class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000074 · EV0000075 · EV0000076 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Function_Call_Definition_Variable_arguments transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Function_Call_Definition_Variable_arguments transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Function_Call_Definition_Variable_arguments transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_02/AV_Summaries/051_Function_Call_Definition_Variable_arguments__33f527b86ff6abf2.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 3 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00014	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...
SRC00015	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00016	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...
SRC00017	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0060. Sidecar variants compare...

C Revision and retrieval practice